

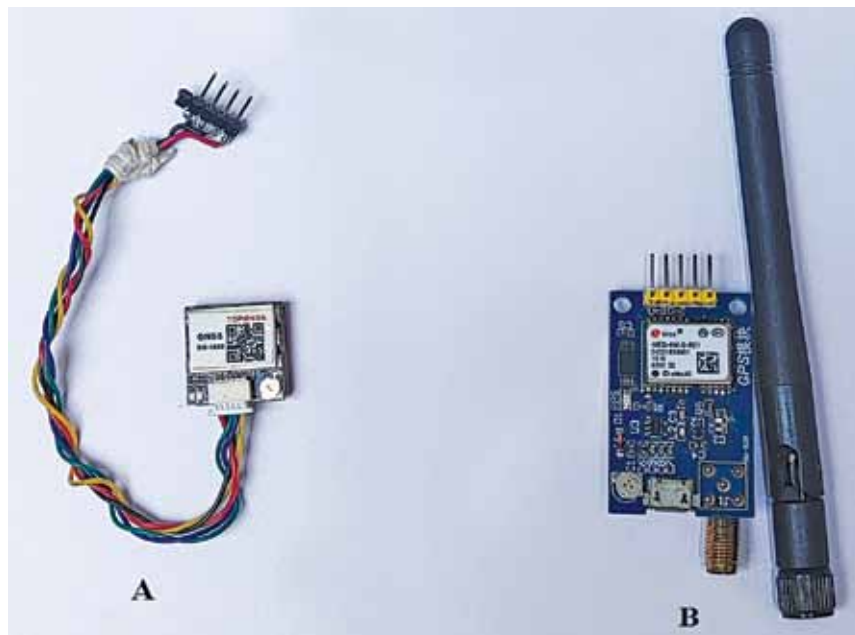


Fig. 3: GPS modules

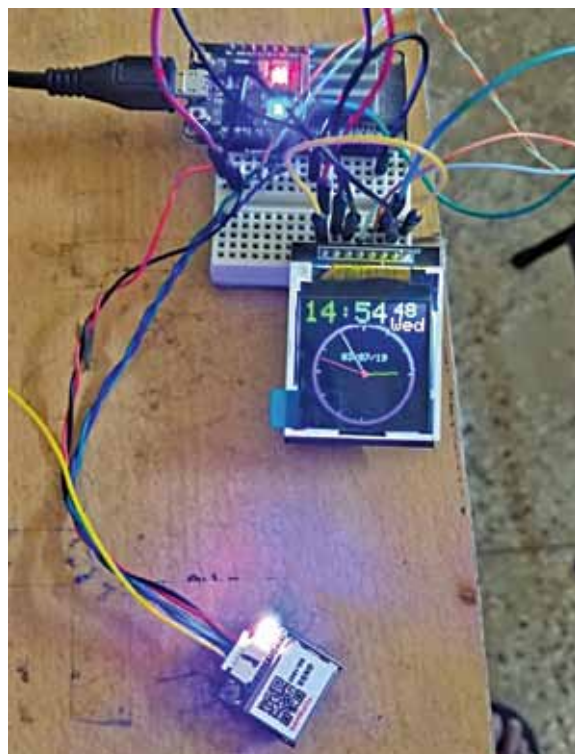


Fig. 4: Author's prototype

invoking the hardware serial of ESP32 and getting the data in CSV format, the following commands helps:

```
Serial2.begin(9600, SERIAL_8N1, RXD2,
TXD2); //RXD2=16, TXD2=17
while (Serial2.available() ) {
String val +=Serial2.readStringUntil('\n');
```

```
r\n');
}
```

That's the few lines of commands in Arduino, which plucks out the comma separated NEMA sentences out of the whole lot.

Clock

Well, till now we have got the CSV strings of the NMEA data and now we have to segregate them and then use for our purpose. We can develop a small GPS clock on ESP32 using this procedure. Once you master this trick, you can connect any UART device to any microcontroller using following commands:

```
val.trim();
len=val.length();
Serial.println(val);
j=val.lastIndexOf(',');
```

```
delay(500);
//Serial.println(j);
//for(i=0;i<=j-1;i++) list[i]= getValue
(val,',',i); // un necessary loads memory
for(i=0;i<=200;i++) list[i]=
getValue(val,',',i); // 200 variable is
enough!
```

The data once acquired in the

string 'val' is now to be stripped in comma-separated array variable named list[]. The NEMA strings happen to be quite long and collecting so much data is generally unnecessary, and it overloads the memory. So, it is quite prudent to restrict the number of CSV variables to 200 or less. Therefore, the next line of the program is used for this purpose.

Local time

After getting the handful CSV values in list[] variable, we can start our local calculations. First, we set an active or fix block to ensure that the GPS receiver is ready and active through following commands. Two variables defined at the top of the program are to offset for the local time. list[1] is time, list[2] is active when it is 'A' and so are the other important variables. The details of the sketch can be followed very easily.

```
if((list[0]=="$GPRMC" or list[0]=="$GNRMC")
&& (list[2]=="A" or list[2]=="V") ) {
//$GNRMC+A or $GPRMC+V

const int timezonehh = 5; //Timezone hour
offset

const int timezonemm = 30; //Timezone
minute offset
```

We may use a TFT colour display, which is entirely controlled by the TFT_ILI9163C.h header file. Any other display with suitable header files can also be used instead.

The prototype is shown in Fig. 4. The circuit diagram of ESP32 based GPS receiver is shown in Fig. 5. It is recommended to use an external 3.7V battery power source for the circuit.

The TFT ILI9163 display works on five pins other than the VCC, LED, and GND pins. SCK and SDA pins of the display, which are fixed type, are connected to GPIO18 and GPIO23 pins of ESP32 board, respectively. The A0, CS, and RESET pins can be connected to any GPIO pins but should be defined in the code accordingly. Pin details of TFT ILI9163C display are shown in Fig. 6.

For finding out the day-of-week, the small function *calcDayOfWeek(int d, int m, int y)* is used in the code (esp32_GPSrx). It finds out the day-of-week from the dd, mm, and yy values. It's so easy!